Address Mapping Calculation Sheet

Logical-to-Physical Address Translation using Relocation, Base and Limit Registers

1. Given Data

Quantity	Value	Source
Logical address (LA)	346	Generated by the CPU
Relocation register (= Base)	14,000	Given in the problem
Limit register	1,000	Assumed (process size); not given

Note: the question gives no limit value, so 1,000 is assumed. The method is identical for any limit.

2. Formula

```
Physical Address (PA) = Logical Address (LA) + Relocation Register
Valid if: Base <= PA < Base + Limit (equivalently 0 <= LA < Limit)
```

3. Step-by-Step Calculation

Step 1: The CPU generates logical address LA = 346.
Step 2: The MMU adds the relocation register:

```
PA = 346 + 14,000 = 14,346
```

Step 3: Compute the upper boundary of the process:

```
Base + Limit = 14,000 + 1,000 = 15,000
```

Step 4: Protection check:

```
14,000 <= 14,346 ? YES
14,346 < 15,000 ? YES
```

Step 5: Result. The address is VALID, no trap occurs, and logical address 346 maps to **physical address 14,346**.

4. Practical Verification (simulated in Python)

A small program mimicking the MMU (`pa = la + 14000; valid = 14000 <= pa < 15000`) was run on several test addresses:

Logical addr	Physical addr (LA+14000)	Within 14000..14999?	Result
346	**14,346**	**Yes**	**VALID**
0	14,000	Yes	VALID
999	14,999	Yes	VALID
1,000	15,000	No	TRAP to OS
1,500	15,500	No	TRAP to OS
-5	13,995	No	TRAP to OS

The first row (346 → 14,346) matches the hand calculation. Addresses 1,000 and above, or negative, are outside the process space and are trapped.

5. Conclusion

Logical address **346** with relocation register **14,000** gives physical address **14,346**, which lies inside the legal range [14,000, 15,000). The MMU performs this translation and the protection check in hardware. See *protection_check.png* for the flowchart and *memory_unit_role.txt* for the MMU's role.